

Core Language Working Group "tentatively ready" Issues for the June, 2016 (Oulu) meeting

Section references in this document reflect the section numbering of document [WG21 N4582](#).

1861. Values of a bit-field

Section: 9.6 [class.bit] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2014-02-13

The resolution of issue 1816 left many aspects of bit-fields unspecified, including whether a signed bit field has a sign bit and the meaning of the bit-field width. Also, the requirement in 3.9.1 [basic.fundamental] paragraph 1 that

For narrow character types, all bits of the object representation participate in the value representation.

should not apply to oversized character-typed bit-fields.

Notes from the June, 2014 meeting:

CWG decided to address only the issue of oversized bit-fields of narrow character types at this time, splitting off the more general questions regarding bit-fields to issue 1943.

Proposed resolution (April, 2016):

Change 3.9.1 [basic.fundamental] paragraph 1 as follows:

...For narrow character types, all bits of the object representation participate in the value representation. **[Note: A bit-field of narrow character type whose length is larger than the number of bits in the object representation of that type has padding bits; see 9.6 [class.bit]. —end note]** For unsigned narrow character types, each possible bit pattern of the value representation represents a distinct number...

2022. Copy elision in constant expressions

Section: 5.20 [expr.const] **Status:** tentatively ready **Submitter:** Jason Merrill **Date:** 2014-10-16

Consider the following example:

```
struct A {
    void *p;
    constexpr A(): p(this) {}
};

constexpr A a;                // well-formed
constexpr A b = A();          // ?
```

The declaration of `a` seems well-formed because the address of `a` is constant.

The declaration of `b`, however, seems to depend on whether copy elision is performed. If it is, the declaration is the equivalent of `a`; if not, however, this creates a temporary and initializes `p` to the address of that temporary, making the initialization non-constant and the declaration ill-formed.

It does not seem desirable for the well-formedness of the program to depend on whether the implementation performs an

optional copy elision.

Notes from the November, 2014 meeting:

CWG decided to leave it unspecified whether copy elision is performed in cases like this and to add a note to 12.8 [class.copy] to make clear that that outcome is intentional.

Notes from the May, 2015 meeting:

CWG agreed that copy elision should be mandatory in constant expressions.

Proposed resolution (April, 2016):

1. Change 5.20 [expr.const] paragraph 1 as follows:

...Expressions that satisfy these requirements, **assuming that copy elision is performed**, are called *constant expressions*. [Note:...

2. Change 7.1.5 [dcl.constexpr] paragraph 7 as follows, breaking the existing running text into a bulleted list:

A call to a constexpr function produces the same result as a call to an equivalent non-constexpr function in all respects except that

- a call to a constexpr function can appear in a constant expression **(5.20 [expr.const]) and**
- **copy elision is mandatory in a constant expression (12.8 [class.copy]).**

3. Change 12.8 [class.copy] paragraph 31 as follows:

...This elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- ...

Copy elision is required where an expression is evaluated in a context requiring a constant expression (5.20 [expr.const]) and in constant initialization (3.6.2 [basic.start.static]). [Note: Copy elision might not be performed if the same expression is evaluated in another context. —end note] [Example:

```
class Thing {
public:
    Thing();
    ~Thing();
    Thing(const Thing&);
};
```

```
Thing f() {
    Thing t;
    return t;
}
```

```
Thing t2 = f();
```

```
struct A {
    void *p;
    constexpr A(): p(this) {}
};
```

```
constexpr A a;           // well-formed, a.p points to a
constexpr A b = A();     // well-formed, b.p points to b
```

```
void g() {
    A c = A();           // well-formed, c.p may point to c or to an ephemeral temporary
}
```

Here the criteria for elision can be combined...

The resolution of issue 1467 made some plausible constructs ill-formed. For example,

```
struct A { A(int); };
struct B { B(A); };
B b{{0}};
```

This is now ambiguous, because the text disallowing user-defined conversions for B's copy and move constructors was removed from 13.3.3.1 [over.best.ics] paragraph 4. Another example:

```
struct Params { int a; int b; };
class Foo {
public:
    Foo(Params);
};
Foo foo{{1, 2}};
```

This is now ambiguous between `Foo(Params)` and `Foo(Foo&&)`.

For non-class types, we allow initialization from a single-item list to perform a copy only if the element within the list is not itself a list (13.3.3.1.5 [over.ics.list] bullet 9.1). The analogous rule for this case would be to add back the bullet in 13.3.3.1 [over.best.ics] paragraph 4, but only in the case where the initializer is itself an initializer list:

the second phase of 13.3.1.7 [over.match.list] when the initializer list has exactly one element **that is itself an initializer list**, where the target is the first parameter of a constructor of class *x*, and the conversion is to *x* or reference to (possibly cv-qualified) *x*,

Proposed resolution (March, 2016):

Change 13.3.3.1 [over.best.ics] paragraph 4 as follows:

...and the constructor or user-defined conversion function is a candidate by

- 13.3.1.3 [over.match.ctor], when the argument is the temporary in the second step of a class copy-initialization, **or**
- 13.3.1.4 [over.match.copy], 13.3.1.5 [over.match.conv], or 13.3.1.6 [over.match.ref] (in all cases), **or**
- **the second phase of 13.3.1.7 [over.match.list] when the initializer list has exactly one element that is itself an initializer list, and the target is the first parameter of a constructor of class *x*, and the conversion is to *x* or reference to (possibly cv-qualified) *x*,**

user-defined conversion sequences are not considered. [Note:...

2091. Deducing reference non-type template arguments

According to 14.8.2.5 [temp.deduct.type] paragraph 17,

If *P* has a form that contains `<i>`, and if the type of the corresponding value of *A* differs from the type of *i*, deduction fails.

This gives the wrong result for an example like:

```
template<int &> struct X;
template<int &N> void f(X<N>&);
int n;
void g(X<n> &x) { f(x); }
```

Here, *P* is `X<N>`, which contains `<i>`. The type of *i* is `int&`. The corresponding value from *A* is *n*, which is a glvalue of type `int`. Presumably this should be valid.

I think this rule means to say something like,

If *P* has a form that contains `<i>`, and the type of *i* differs from the type of the corresponding template parameter of the template named by the enclosing *simple-template-id*, deduction fails.

Proposed resolution (March, 2016):

Change 14.8.2.5 [temp.deduct.type] paragraph 17 as follows:

If P has a form that contains $\langle i \rangle$, and if the type of i differs from the type of the corresponding template parameter of the template named by the enclosing *simple-template-id*, deduction fails. If P has a form that contains $[i]$...

2137. List-initialization from object of same type

Section: 8.5.4 [dcl.init.list] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-06-10

It is not clear in code like the following that selecting a copy/move constructor is the correct choice when an initializer list contains a single element of the type being initialized, as required by issue 1467:

```
#include <initializer_list>
#include <iostream>

struct Q {
    Q() { std::cout << "default\n"; }
    Q(Q const&) { std::cout << "copy\n"; }
    Q(Q&&) { std::cout << "move\n"; }
    Q(std::initializer_list<Q>) { std::cout << "initializer list\n"; }
};

int main() {
    Q x = Q { Q() };
}
```

Here the intent is that Q objects can contain other Q objects, but this is broken by the resolution of issue 1467.

Perhaps the presence of an initializer-list constructor should change the outcome?

Proposed resolution (April, 2016):

1. Change 8.5.4 [dcl.init.list] bullet 3.1 as follows:

List-initialization of an object or reference of type τ is defined as follows:

- If τ is a class type **an aggregate class** and the initializer list has a single element of type $cv\ u$, where u is τ or a class derived from τ , the object is initialized from that element (by copy-initialization for copy-list-initialization, or by direct-initialization for direct-list-initialization).
- ...

2. Change 13.3.3.1.5 [over.ics.list] paragraph 2 as follows:

If the parameter type is a **an aggregate** class x and the initializer list has a single element of type $cv\ u$, where u is x or a class derived from x , the implicit conversion sequence is the one required to convert the element to the parameter type.

3. Change 13.3.3.1.5 [over.ics.list] paragraph 6 as follows, breaking the existing running text into a bulleted list:

Otherwise, if the parameter is a non-aggregate class x and overload resolution per 13.3.1.7 [over.match.list] chooses a single best constructor c of x to perform the initialization of an object of type x from the argument initializer **list, list:**

- **If c is not an initializer-list constructor and the initializer list has a single element of type $cv\ u$, where u is x or a class derived from x , the implicit conversion sequence has Exact Match rank if u is x , or Conversion rank if u is derived from x**
- **Otherwise**, the implicit conversion sequence is a user-defined conversion sequence with the second standard conversion sequence an identity conversion.

If multiple constructors are viable...

2145. Parenthesized declarator in function definition

Section: 8.4.1 [dcl.fct.def.general] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-06-19

According to 8.4.1 [dcl.fct.def.general] paragraph 2,

The *declarator* in a *function-definition* shall have the form

D1 (*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*ref-qualifier*_{opt} *exception-specification*_{opt} *attribute-specifier-seq*_{opt} *trailing-return-type*_{opt}

However, in practice implementations accept a parenthesized declarator in a function definition.

Proposed resolution (April, 2016):

Change 8.4.1 [dcl.fct.def.general] paragraph 2 as follows:

~~The *declarator* in~~ In a *function-definition* ~~shall have the form,~~

~~D1 *parameters-and-qualifiers* *trailing-return-type*~~_{opt}

either *void declarator* ; or *declarator* ; shall be a well-formed function declarator as described in 8.3.5 [dcl.fct].

A function shall be defined only in namespace or class scope.

2171. Triviality of copy constructor with less-qualified parameter

Section: 12.8 [class.copy] **Status:** tentatively ready **Submitter:** Jason Merrill **Date:** 2015-09-14

Issue 1333 says that a defaulted copy constructor with a less-const-qualified parameter type than the implicit declaration is non-trivial. This is inconsistent with the usual pattern that whether a special member function is callable is separate from whether it is trivial; the different declaration only affects whether you can call it with a `const` argument, it doesn't affect the operations involved. Should this outcome be reconsidered?

Proposed resolution (April, 2016):

1. Change 12.8 [class.copy] paragraph 12 as follows:

A copy/move constructor for class *x* is trivial if it is not user-provided, ~~its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration,~~ and if...

2. Change 12.8 [class.copy] paragraph 25 as follows:

A copy/move assignment operator for class *x* is trivial if it is not user-provided, ~~its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration,~~ and if...